Computer Architecture 2025

Exercise Session Report

Group: 36

Name and student number:

Pengyu Zhang	r1016626
Binkai Chen	r1016447

Date: 02/05/2025

Part 1: Fill in the Design Metrics

Implementation	*Simulation*	*Backend*						
		Synthesis				*Floorplan*	*Signoff*	
	*# of Cycles for MULT**	*Design area (μm^2)*	*Critical path (ns)*	*Maximum operating frequency (MHz)*	*Minimal time to execute the MULT (ns)*	*Die area (μm^2)*	*Area utilization (%)*	*Maximum operating frequency (MHz)*
*1. Single Cycle***	*2300*	*508266*	*9.37*	*53.4*	*43071.1*	***		
2. Single Cycle with Multiplication	*40*	*616669*	*11.83*	*42.3*	*945.6*			
3a. Basic Pipelined	*40*	*637006*	*0.88*	*568.2*	*70.4*	*1864506.9*	*36*	*195.3(2.56ns)*
3b. Basic Pipelined (50MHz, AREA 2) BEST AREA	*40*	*633033*	*14.78*	*67.7*	*590.8*	*1853013.7*	*36*	*67.1(14.90ns)*
3c. Basic Pipelined (50MHz, AREA 3) BEST DELAY	*40*	*718938*	*0.66*	*757.6*	*52.8*	*2101354.918*	*33*	*202.4(2.47ns)*
4. Pipelined with hazard logic	*25*	*642159*	*0.88*	*568.2*	*44.0*			
5. Advanced acceleration	*828*	*641967*	*0.92*	*543.5*	*1523.5*			

** The program*

- *MULT1 is for “1. Single cycle”;*
- *MULT2 is for “2. Single Cycle with Multiplication” and “3a/3b. Basic Pipelined”;*
- *MULT3 is for “4. Pipelined with hazard logic”;*
- *MULT4 (or your modified version of it) is used for “5. Advanced acceleration”.*

*** Implementations 1, 2, 3a, 4 and 5 have a clock frequency of 10MHz and should be synthesized under the default strategy preset “AREA 0”.*

Presets 1 & 2 must be replaced by your synthesis strategy after performing the Synthesis Exploration step (e.g., DELAY 0-4/AREA 0-3)

**** The grey regions are not required to fill in.*

Part 2: Answer the Questions based on Your Designs

Questions:

1. *What is the critical path of the single-cycle processor (simple program)? Which operation does this critical path stand for?*

   The critical path for this processor is 9.37 ns. From the Instruction memory to the data memory address port passing through the ALU. This path is used by load and store command.

2. *For the single-cycle processor, what are the top-5 components in terms of area? After adding MULT-support, what are the top-5 components then? What has the changed? What is your explanation for this?*

   Single-cycle processor: 1. sram_BW64 (44.8%, 227869) 2. sram_BW32 (27.3%, 138703) 3. Register file (23.4%, 119126) 4. alu (3.6%, 18458) 5. Branch unit (0.8%, 4116)

   Single-cycle processor with multiplication: 1. sram_BW64 (40.0%, 227869) 2. sram_BW32 (22.5%, 138703) 3._alu (20.6%, 126769) 4._register_file (19.3%, 119157) 5._branch_unit (0.7%, 4116)

   Analysis and Explanation: It's obvious that the alu occupies more area after adding MULT-support, which is quite reasonable as we mainly modify the alu.v and alu control.v to add MULT-support.

3. *What is the critical path after you support the MULT operation in the single-cycle processor? Is there anything changed? What is your explanation for this?*

   Critical path increased to 11.83 ns. From instruction memory to data memory and passing through ALU for the calculation of the data address. Possible cause is the increased size of ALU which squeezed other logic units. Causing the wiring to be more difficult.

4. *Regarding to your findings in Q2 & Q3, do you think adding MULT-operator support is always a good choice for any microprocessor? Please elaborate your answer.*

   Since adding single cycle multiplication will increase the size of the ALU by almost 6 times, and total area by 20%, it is not recommended to do so for area sensitive design. However, it is also clear that adding multiply will decrease the total calculation time considerably. So, for performance sensitive design, multiplication is recommended.

5. *What is the critical path of your basic pipelined design? How does the critical path length change compare to the single-cycle version? What about your final advanced version? Try to discuss the benefits and costs of processor pipelining according to your findings (hint: speed/throughput/area/power/etc.).*

   Basic pipelined design: The critical path is 0.88 ns in Synthesis and 2.56 ns id signoff. It goes from EX/MEM pipeline register to data memory. Compared to the single-cycle processor, the critical path decreased drastically.

   Advanced pipeline design: The critical path is still 0.88ns. It goes from EX/MEM pipeline register to data memory.

   Benefits and costs of pipelining: Pipelining can substantially reduce the critical path by inserting registers, enabling higher clock frequency and larger throughput. The area overhead is larger since we employ pipeline registers and hazard-handling units here. Due to the same reason, the power consumption is also larger.

6. *What techniques have you applied to accelerate MULT4? Explain under what conditions/workloads your*

strategies would have the maximum/minimum performance gain. What is the cost for your strategy?

For the hardware part, additional forwarding and data hazard detection have been added for the beq instruction. This helps save cycles if the data required by beq has not yet been written back.

We applied loop unrolling on dimension B and C to accelerate MULT4. This strategy is effective when the loop size is relatively small and fixed. When the loop size is dynamic or very large then loop unrolling would be impractical. The major cost of loop unrolling is that there will be huge code size increment, making it difficult for maintenance and further adjustment.

7. *(Backend Flow) Please explain the following terms of the backend flow: PDK, LEF, CTS, STA, GDSII. Which PDK are we using?*

   PDK stands for Process Design Kit. It describes the electric property of physical devices as well as the relation between the device parameters in the EDA tools and the physical device dimensions. The PDK we are using is SKY130.

   LEF stands for Library Exchange Format. It describes the dimension and port positions of basic units. But the inner structure of the units are not covered. LEF is used for placement and routing.

   CTS stands for Clock Tree Synthesis. It's a step which clock delivering network is generated.

   STA stands for Static Timing Analysis. It's a step in which timing of the design will be checked. Including Setup Time and Hold Time.

   GDSII refers to a Graphic Data System II file. It contains the final Geometric parameters of the physical chip. Including the positions and dimensions of the transistors, vias and wires. This file will later be delivered to the foundry to tape out.

8. *(Backend Flow) Have you encountered any warnings/errors in the backend session? What are they? How do you fix those critical issues?*

   There are 3 warnings that we encountered: max slew violations, max fanout violations and max capacitance violations. According to the report, the first warning means some nodes have a slew rate that is too high. Which means the signal changes fast. The second warning means some outputs are driving too many inputs. And the last warning means the capacitance on some node is too large. Which may cause problems on delay. However, none of these warnings are fatal thus we didn't fix them. Overall, the major limitation that affects our progress severely is the limited storage of our ESAT accounts. Given the tiny 10G storage, we often ran into disk quota exceeding warnings and had to free up storage space to continue.

9. *(Backend Flow) The major steps of this session's backend flow are Synthesis, Floorplan, Placing and Routing. Have a look at the "runs/results/" folder. What output files does each step generate? Anything similar/different between steps? What are the later steps doing upon the previous?*

   Synthesis: This stage generates .v file which contains the results after breaking the original design into small basic units. Also, .sdf file which describes the delays of each interconnection is generated.

   Floorplan: This stage generate .def file which contains the position of each standard cells. Another .odb file is the standard file format of OpenLane. It contains more detailed information and can be inspected by corresponding tools.

   Placing: This stage generates a .odb file which can be opened by EDA tools to visualize the chip. A .nl.v file which contains gate-level netlist. A .pnl.v file that contains netlist after placement. A .def file that contains the position of each standard units.

   Routing: This step generates .def, .nl.v, .pnl.v, and .odb files, which serve a similar role as in placement, but the results are post-routing.

   Once we finish the previous steps, we run the signoff to get the final GDSII design file.

10. *(Backend Flow) Try to compare between the three full-flow scenarios. What is the impact of the target clock frequency? What is the impact of different synthesis strategy presets? Please discuss the tradeoff in terms of speed and area, i.e. if not using AREA-0, which preset will you choose after the synthesis exploration and why?*

    Different presets will instruct the backend program to try different ways of implementation but aiming in different directions. Higher target frequency will require a lower critical path. In the case of the best area, a better area will be obtained at the cost of longer critical path. However, the program will try to keep the critical path within the maximum time before optimizing further. For best delay optimization, shorter critical path will be obtained but area could be much larger. In this design, we would prefer best delay(area 3) implementation for it increased speed considerably but the area does not increased a lot.

11. *(Optional survey) Have you used LLM services to help with this project (as suggested in Session1-Page8)? If so, which model/service? What the task have you submitted? To which extent do you find it helpful? Or, does it misguide your exploration? What is your general opinion/suggestion on using LLMs in this course?*

    We used ChatGPT to help with the project. We asked ChatGPT for some background information and employed it to debug our RTL. Overall, ChatGPT is of great help to us, and we think it's a great idea to encourage using LLMs in this course since they can solve almost all the minor bugs and help us to focus more on the core content.

Part 3: Post your design photos

*Following the step "**Analysis and Discussion #5**" in Jupyter Lab, take a screenshot of the final die photo. If the signoff is unavailable, post the photo from the furthest stage you can reach with your RTL and mention the stage of it.*

Photo 1: Signoff - Basic Pipelined (10MHz)

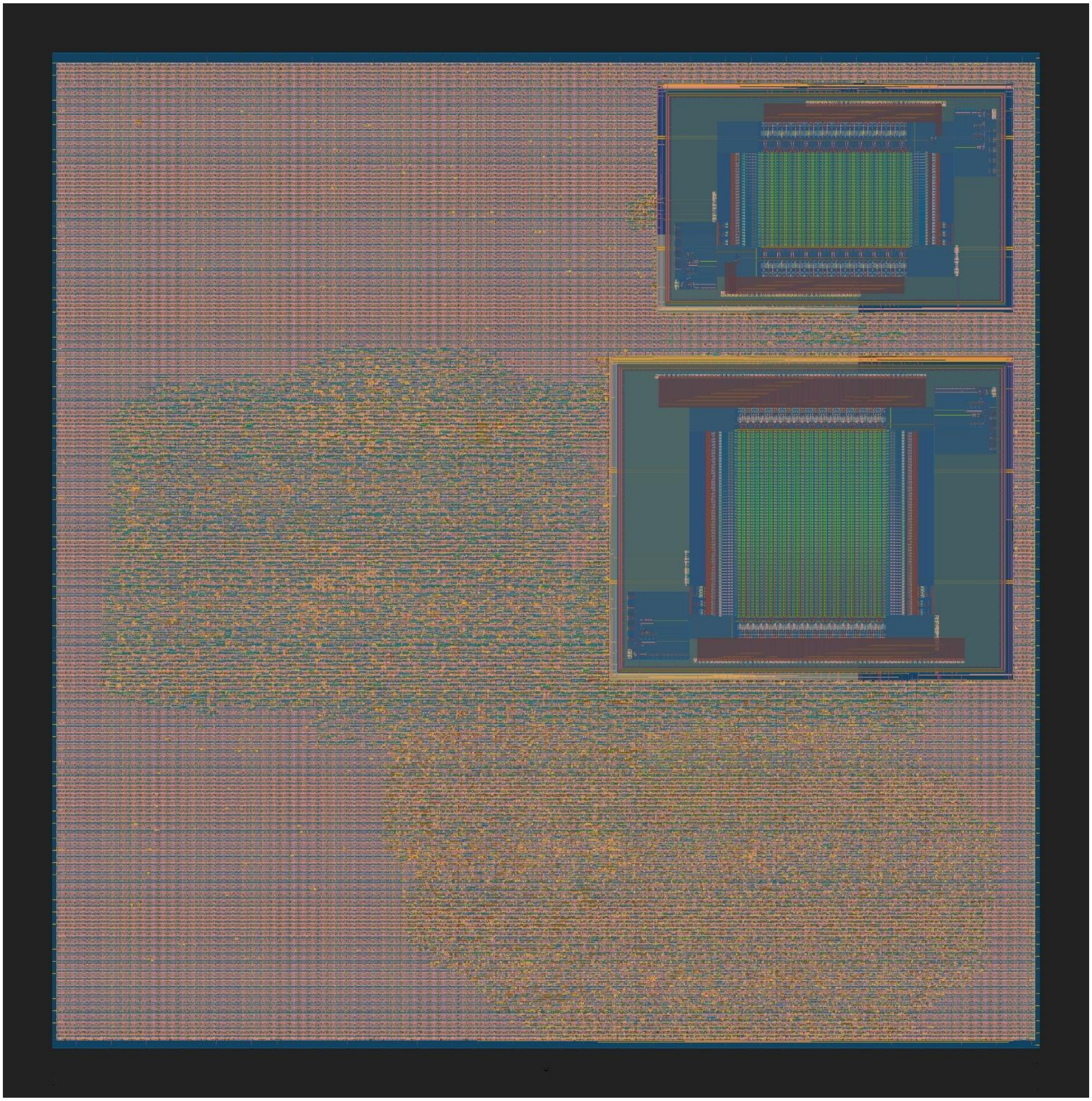

Figure 1 Die photo of basic pipelined version

Photo 2: Signoff - Basic Pipelined (50MHz)

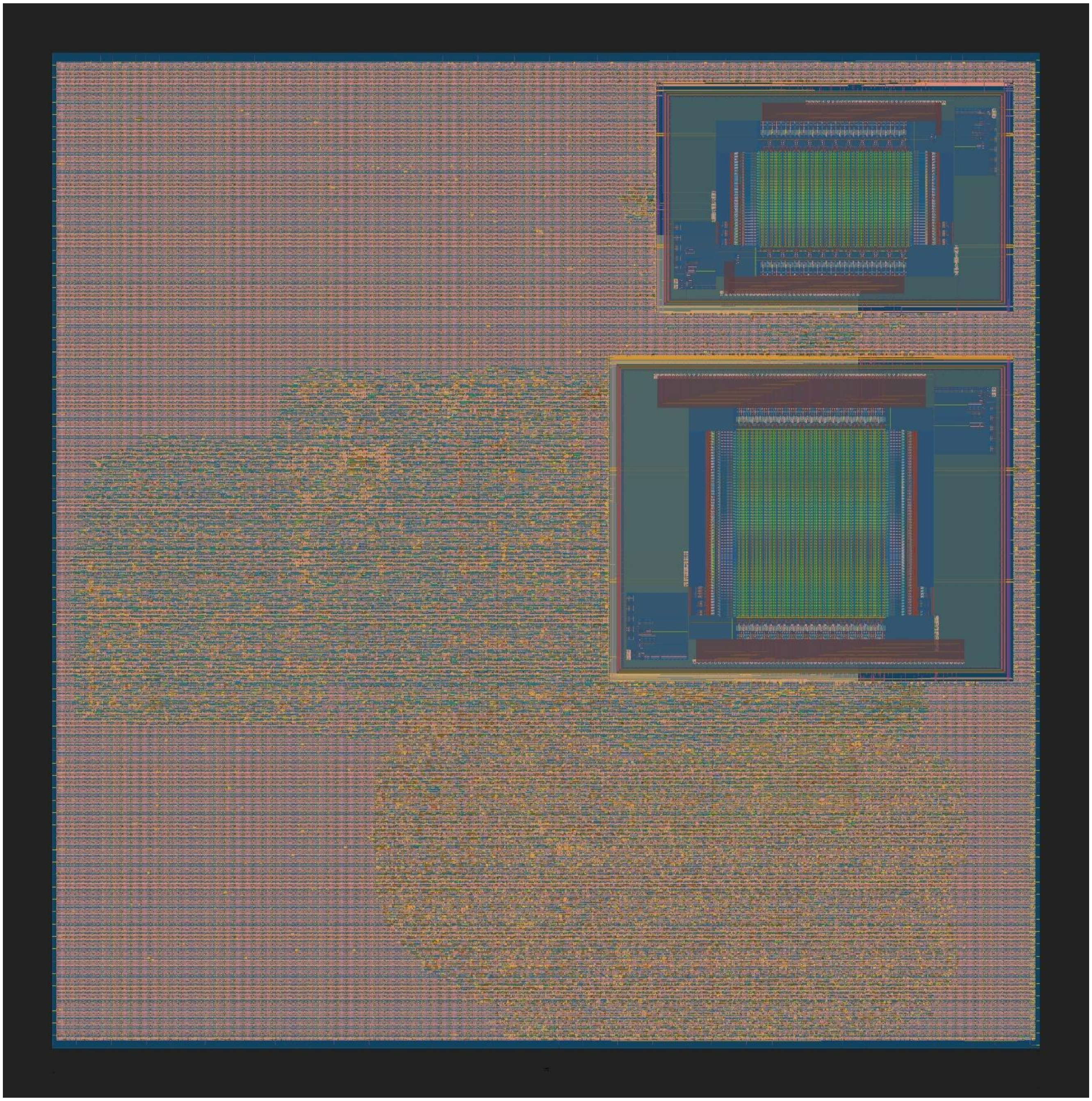

Figure 2 Die photo of basic pipelined version, using the best area strategy (Area 2)

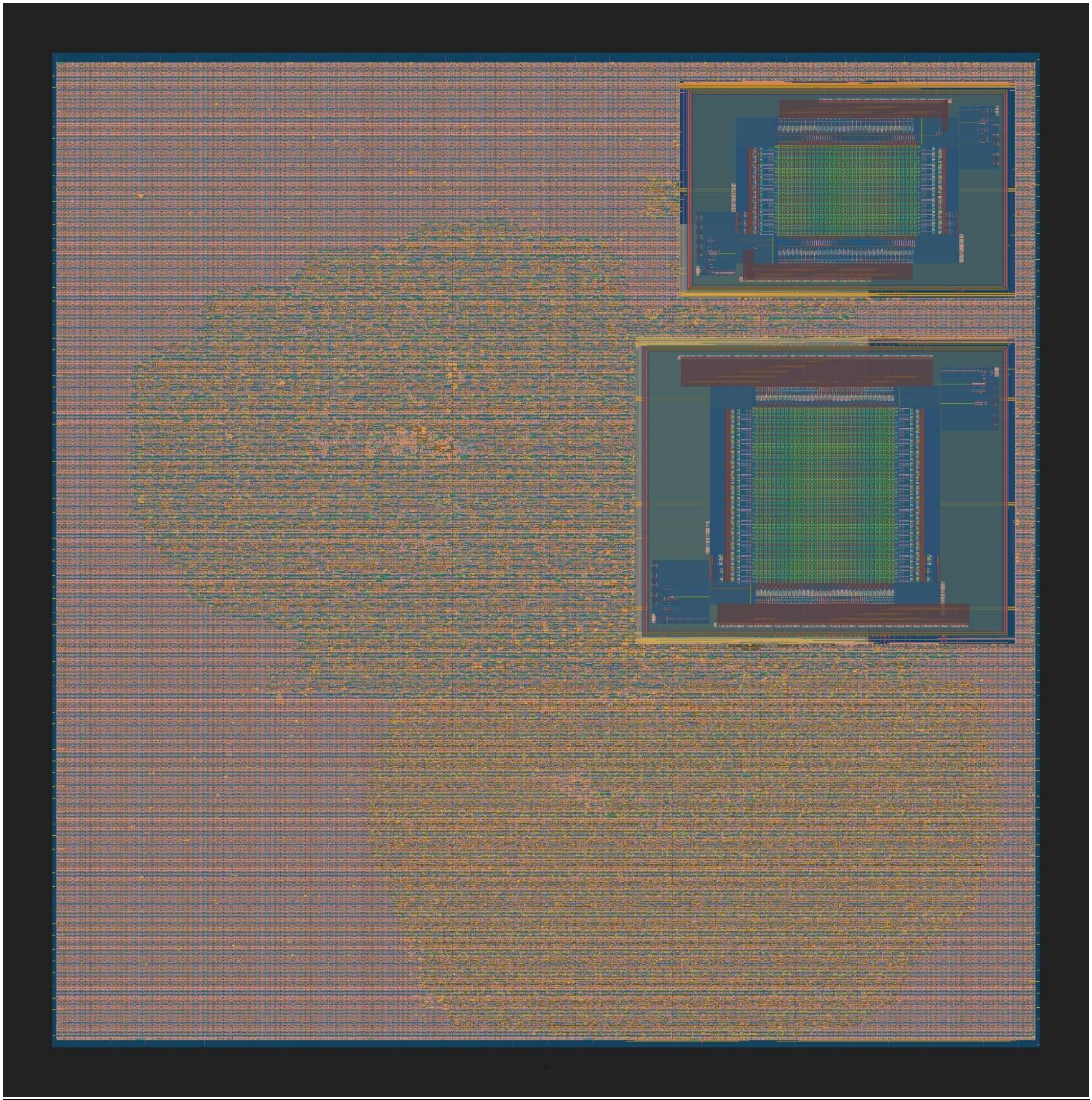

Figure 3 Die photo of basic pipelined version, using best delay strategy (Area 3)